Lab Title: Structural Design Patterns Using a Food Delivery System

Objectives

By the end of this lab, students should be able to:

- Understand the purpose of structural design patterns.
- Implement the **Adapter** pattern to make incompatible interfaces work together.
- Implement the **Decorator** pattern to add responsibilities dynamically.
- Use the **Facade** pattern to provide a simple interface to a complex subsystem.
- Implement the **Composite** pattern to represent hierarchical structures.
- Understand the **Proxy** pattern for controlling access to an object.
- Select an appropriate structural pattern based on a software requirement.

Introduction to Structural Design Patterns

Structural design patterns focus on **how classes and objects are combined to form larger and more flexible structures**.

In this lab, we will use one **Food Delivery System** throughout all five patterns.

The system contains:

- Customers
- Food items
- Restaurants
- Orders
- Payments
- Delivery services
- Menus

We will gradually improve the same system using different structural patterns.

1. Adapter Pattern

1.1 Purpose

The **Adapter Pattern** allows two incompatible interfaces to work together. For example, our Food Delivery System expects a payment service with:

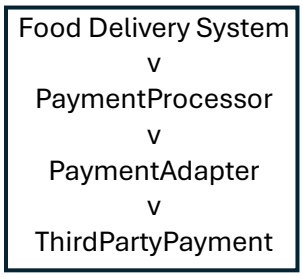
```
pay(double amount)
```

However, an existing third-party payment service provides:

```
makePayment(double amount)
```

Instead of changing our existing system or the third-party service, we create an **Adapter** between them.

1.2 Structure



1.3 Implementation

Target Interface

```
interface PaymentProcessor {
void pay(double amount);
}
```

Existing Third-Party Class

```
class ThirdPartyPayment {
  public void makePayment(double amount) {
    System.out.println("Payment of $" + amount + " processed.");
  }
}
```

Adapter

```
class PaymentAdapter implements PaymentProcessor {
  private ThirdPartyPayment paymentService;
  public PaymentAdapter(ThirdPartyPayment paymentService) {
    this.paymentService = paymentService;
  }
  @Override
  public void pay(double amount) {
    paymentService.makePayment(amount);
  } }
```

Client

```
public class Main {
  public static void main(String[] args) {
    ThirdPartyPayment thirdParty = new ThirdPartyPayment();
    PaymentProcessor processor =
        new PaymentAdapter(thirdParty);
    processor.pay(2500);
  }
}
```

1.4 Task

Modify the example to support another existing payment service:

```
class MobileWallet {
  public void sendMoney(double amount) {
    System.out.println("Money sent: $" + amount);
  }
}
```

Create an adapter so that MobileWallet can also be used through: **PaymentProcessor**

2. Decorator Pattern

2.1 Purpose

The **Decorator Pattern** allows us to add new functionality to an object **without modifying its original class**. Suppose a customer orders: Burger

They can add:

+ Cheese

+ Extra Sauce

+ Fries

Instead of creating separate classes such as:

- CheeseBurger
- SauceBurger
- CheeseSauceBurger
- CheeseSauceFriesBurger

we can dynamically decorate the original food item.

2.2 Structure

Food

+-- Burger

+-- FoodDecorator

    +-- CheeseDecorator

    +-- SauceDecorator

2.3 Implementation

Component

```
interface Food {
   String getDescription();
   double getPrice();
}
```

Concrete Component

```
class Burger implements Food {
   @Override
   public String getDescription() {
      return "Burger";
   }
   @Override
   public double getPrice() {
      return 500;
   }
}
```

Decorator

```
abstract class FoodDecorator implements Food {
  protected Food food;
  public FoodDecorator(Food food) {
    this.food = food;
  }
}
```

Cheese Decorator

```
class CheeseDecorator extends FoodDecorator {
  public CheeseDecorator(Food food) {
    super(food);
  }
  @Override
  public String getDescription() {
    return food.getDescription() + " + Cheese";
  }
  @Override
  public double getPrice() {
    return food.getPrice() + 100;
  }
}
```

Sauce Decorator

```
class SauceDecorator extends FoodDecorator {
  public SauceDecorator(Food food) {
    super(food);
  }
  @Override
  public String getDescription() {
    return food.getDescription() + " + Extra Sauce";
  }
  @Override
  public double getPrice() {
    return food.getPrice() + 50;
  }
}
```

Client

```
public class Main {
  public static void main(String[] args) {
    Food order = new Burger();
    order = new CheeseDecorator(order);
    order = new SauceDecorator(order);
    System.out.println(order.getDescription());
    System.out.println("Price: $" + order.getPrice());
  }
}
```

2.4 Task

Add two more decorators:

- FriesDecorator → adds $150
- DrinkDecorator → adds $100

Allow the customer to create combinations such as:
Burger + Cheese + Fries + Drink

3. Facade Pattern

3.1 Purpose

The **Facade Pattern** provides a **simple interface to a complex subsystem**. Placing a food order may require several operations:

```
Check Restaurant
      ↓
Prepare Food
      ↓
Process Payment
      ↓
Arrange Delivery
      ↓
Send Confirmation
```

Without a Facade, the client has to interact with all these classes.
The Facade provides one simple method: placeOrder()

3.2 Subsystems

```
class Restaurant {
  public void prepareFood() {
    System.out.println("Food is being prepared.");
  }
}
```

```
class PaymentService {
  public void processPayment() {
    System.out.println("Payment processed.");
  }
}
```

```
class DeliveryService {
    public void arrangeDelivery() {
        System.out.println("Delivery arranged.");
    }
}
```

```
class NotificationService {
    public void sendConfirmation() {
        System.out.println("Order confirmation sent.");
    }
}
```

3.3 Facade

```
class FoodDeliveryFacade {
    private Restaurant restaurant;
    private PaymentService payment;
    private DeliveryService delivery;
    private NotificationService notification;
    public FoodDeliveryFacade() {
        restaurant = new Restaurant();
        payment = new PaymentService();
        delivery = new DeliveryService();
        notification = new NotificationService();
    }
    public void placeOrder() {
        restaurant.prepareFood();
        payment.processPayment();
        delivery.arrangeDelivery();
        notification.sendConfirmation();
} }
```

3.4 Client

Instead of calling four different services:

```
FoodDeliveryFacade facade = new FoodDeliveryFacade();
facade.placeOrder();
```

The client only needs to know about the **Facade**.

3.5 Task

Add a TrackingService:

```
class TrackingService {
    public void startTracking() {
        System.out.println("Delivery tracking started.");
    }
}
```

Modify the Facade so that tracking starts automatically when an order is placed.

4. Composite Pattern

4.1 Purpose

The **Composite Pattern** is useful when objects need to be organized into a **tree-like hierarchical structure**. A food delivery menu can contain:

```
Menu
 ├── Fast Food
 │    ├── Burger
 │    └── Pizza
 │
 └── Drinks
      ├── Cola
      └── Juice
```

Both individual food items and groups of food items can be treated in the same way.

4.2 Component

```
interface MenuComponent {
    void show();
}
```

4.3 Leaf

```
class FoodItem implements MenuComponent {
    private String name;
    public FoodItem(String name) {
        this.name = name;
    }
    @Override
    public void show() {
        System.out.println("- " + name);
    }
}
```

4.4 Composite

```
import java.util.ArrayList;
import java.util.List;
class MenuCategory implements MenuComponent {
    private String name;
    private List<MenuComponent> items = new ArrayList<>();
    public MenuCategory(String name) {
        this.name = name;
    }
    public void add(MenuComponent item) {
        items.add(item);  }
```

```
    @Override
    public void show() {
      System.out.println(name);
      for (MenuComponent item : items) {
        item.show();
      } } }
```

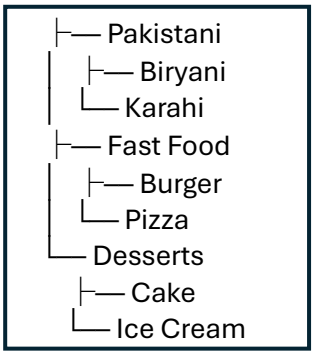
4.5 Client

```
public class Main {
  public static void main(String[] args) {
    MenuCategory menu = new MenuCategory("Main Menu");
    MenuCategory fastFood = new MenuCategory("Fast Food");
    fastFood.add(new FoodItem("Burger"));
    fastFood.add(new FoodItem("Pizza"));
    MenuCategory drinks = new MenuCategory("Drinks");
    drinks.add(new FoodItem("Cola"));
    drinks.add(new FoodItem("Juice"));
    menu.add(fastFood);
    menu.add(drinks);
    menu.show();
  }
}
```

4.6 Task

Create the following menu hierarchy:

Food Delivery Menu



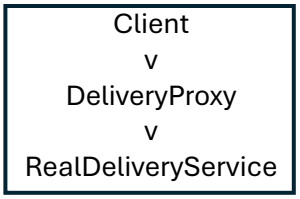
Use the same MenuComponent, FoodItem, and MenuCategory classes.

5. Proxy Pattern

5.1 Purpose

The **Proxy Pattern** provides a substitute object that controls access to another object. In our Food Delivery System, suppose accessing the delivery tracking service is expensive or requires authorization.

Instead of allowing the client to directly access the real service, we use a proxy.



5.2 Subject

```
interface DeliveryService {
   void trackOrder();
}
```

5.3 Real Subject

```
class RealDeliveryService implements DeliveryService {
   @Override
   public void trackOrder() {
      System.out.println("Tracking delivery...");
   }}
```

5.4 Proxy

```
class DeliveryProxy implements DeliveryService {
   private RealDeliveryService realService;
   private boolean authorized;
   public DeliveryProxy(boolean authorized) {
      this.authorized = authorized;
   }
   @Override
   public void trackOrder() {
      if (!authorized) {
         System.out.println("Access denied.");
         return;
      }
      if (realService == null) {
         realService = new RealDeliveryService();
      }
      realService.trackOrder();
   }}
```

5.5 Client

```
public class Main {
   public static void main(String[] args) {
      DeliveryService service = new DeliveryProxy(true);
      service.trackOrder();
   }}
```

The proxy can therefore control access before allowing the client to use the real service.

5.6. Task

Modify the proxy so that:

- Unauthorized customers cannot track orders.
- Authorized customers can track orders.
- The real delivery service is created only when tracking is actually requested.